Malware Obfuscation using Kanerva's Sparse Distributed Memory

Tim Johns

December 2025

1 Introduction

Sparse Distributed Memory (SDM), introduced by Pentti Kanerva, is a theoretical model of associative memory that stores and retrieves information using high-dimensional binary address spaces. This paper explores how SDM-style encoding and retrieval can be adapted as an obfuscation strategy for malware, where payload bytes are not stored contiguously but instead distributed across many "memory locations" and reconstructed only at runtime.

2 What is a Sparse Distributed Memory?

A Sparse Distributed Memory is an associative memory system, meaning it is a system wherein information is stored and retrieved via some form of association, as opposed to a hard-coded location in memory. This concept is a component of neuromorphic computing, which is a field of study that examines computer systems that mimic the biological or architectural structure of the human brain.

2.1 Structure of an SDM

An SDM operates at the software level, rather than the hardware level. The architecture consists of two p-length arrays, each containing a set of n-bit vectors. As we will see, one of these p-length arrays will contain vectors that are all initialized to 0. This is the **data vector**. It will eventually hold the bit patterns for data we store in the SDM. The other array will have vectors initialized to contain a random pattern of ones and zeroes. These are the **memory addresses**. Each vector in the address array maps to a vector in the data array like this:

It is important to note that these address vectors are not sequential addresses, they are randomly filled with ones and zeros. They do not have any relationship to the adjacent memory addresses in the address array. Regarding the number of address-data vector pairs in our SDM, the larger the value we select for p, the more "coverage" we achieve in the n-bit space. It will give us better recall and capacity to store memories in the SDM, but it will also use more physical memory, and take longer to read and write to.

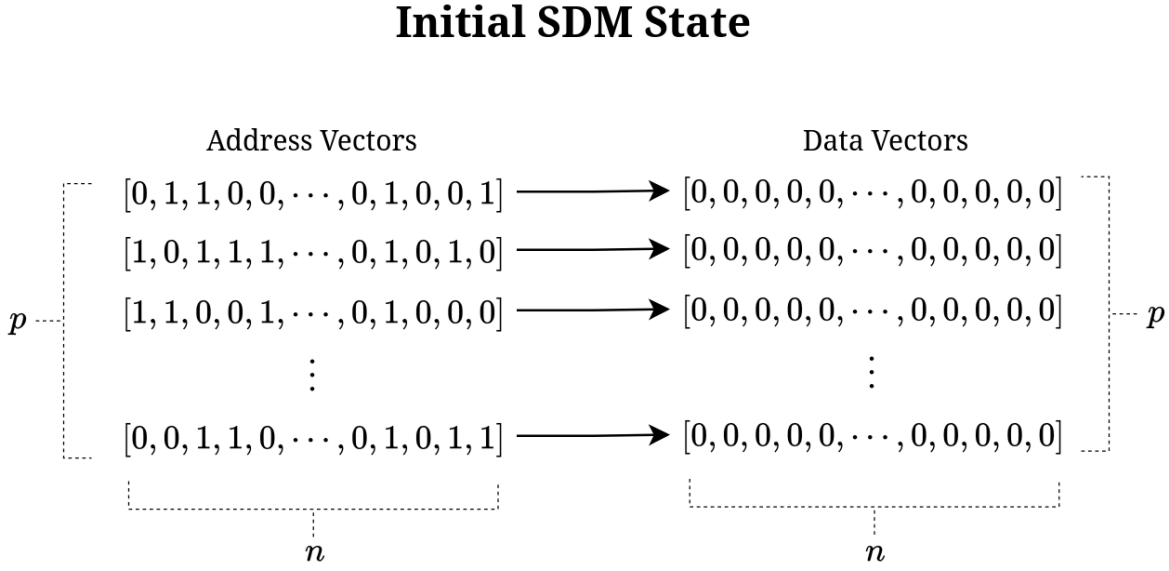

Figure 1: Each address vector corresponds to a data vector, representing it like a hardware memory address.

Regarding the n value we pick, this is where we get the "sparse" attribute of our SDM. Since our SDM architecture functions as base 2, there are 2^n possible address spaces we could use. However, recall that we initialized the address vectors to be full of random patterns of 1's and 0's, rather than sequential iterations starting from $[0, 0, 0, \cdots 0, 0, 0]$ to $[1, 1, 1, \cdots 1, 1, 1]$ (like how traditional hardware memory addressing works). Therefore we do not have 2^n places to store data. Our data storage is *sparse*, as $p < 2^n$. The longer the n value we pick, the cleaner our data retrieval process is likely to be, as there is more space to allow vectors to distringuish themselves from one another.

In code, this is the class I have defined to generate an SDM.

```
class SDM:

    def __init__(self, p, n):
        """
        Constructor that initializes the SDM with random addresses and zeroed data stora
        """
        #Initialize the number of addresses:
        self.p = p
        #Initialize length of addresses:
        self.n = n
```

```
#Initialize array of random addresses:
self.addresses = np.random.randint(0, 2, (p, n))
#Initialize array of data storage with zeros:
self.data = np.zeros((p, n))
#Initialize value for the radius:
self.radius = 0.451 * n
```

2.2 Storing Data

The way that data is stored in the SDM is quite simple. Suppose we have an input vector, v that we want to store in our SDM. Since our memory is **distributed**, we are not just going to store it at the first memory address that we come across. Rather, we are going to store it at **all** the memory address that are sufficiently "close" to the data we are actually storing (this is how we are developing associative memory). To do this, we compute the hamming distance between our input vector and the memory address vector.

Hamming distance. The *Hamming distance* between two equal-length binary vectors $a, b \in \{0,1\}^n$ is the number of bit positions where they differ:

$$\text{ham}(a,b) \;=\; \sum_{j=0}^{n-1} \mathbf{1}[a_j \neq b_j]\,,$$

where $\mathbf{1}[\cdot]$ is 1 if the condition is true and 0 otherwise. Equivalently, it is the number of bit flips needed to transform a into b.

Example. Let $v_1 = [0,1,1,0,0,0,1,0,0,1]$ and $v_2 = [0,0,1,0,0,1,1,0,1,1]$. They differ in positions 2, 6 and 9, so $\text{ham}(v_1, v_2) = 3$.

We determine whether or not we are going to "write" to a particular address based on whether the hamming distance we calculated between the input and address is close enough to a defined radius. **Essentially, if our** *hamming distance* $\leq$ *radius***, we store the input at that address.** As such, the larger the radius, the more addresses we will store a value at.

If you recall, in the code, we initialized our radius like this:

```
#Initialize value for the radius:
self.radius = 0.451 * n
```

We calculate the radius from $0.451 * n$ because, in our base 2 system, distances between vectors cluster around $n/2$. This is the case because, for any pair of bits, there is roughly a 50% chance that they match. So, for an n-bit vector, we'd expect the distance to be roughly $0.5n$. By setting the radius to $0.451n$, we are tightening our neighborhood by requiring the input vector to be closer than random to our address. Again, this is how we are developing associative memory.

To write the actual input vector into the SDM, we simply go bit-by-bit, checking if the current input vector bit is a 1 or a 0. If it is a 1, we add 1 to the corresponding bit in the **data vector**. If the input vector bit is a 0, we subtract 1. This process is repeated for every

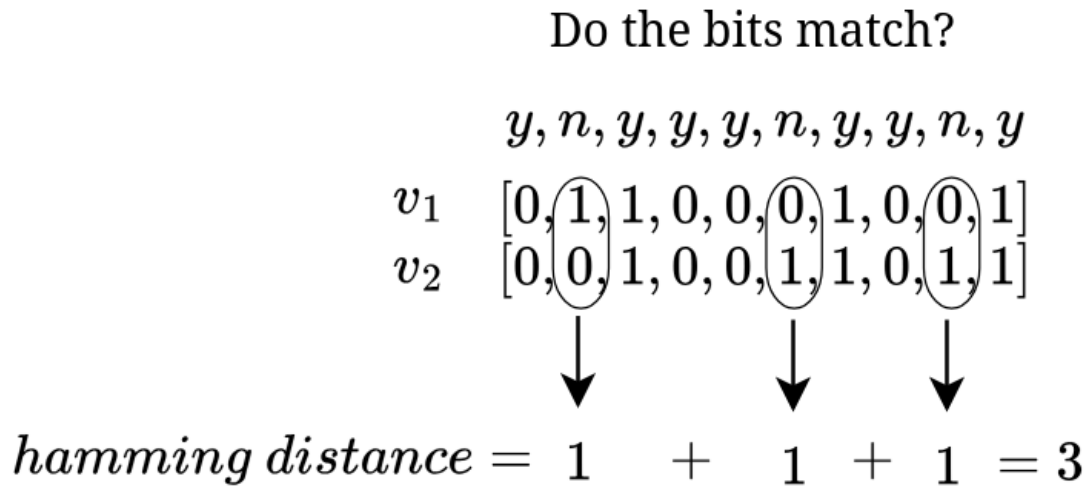

Figure 2: How the Hamming Distance is calculated.

address in the SDM that is "close enough" to the data vector according to our hamming distance comparison.

Here is how that works in our code:

```
def enter(self, addressVector):
    """
    This method 'enters' the info in an
    address vector into the sdm's data array
    by finding all the addresses
    within the radius of the given address
    vector, and then writing
    the given address's info to the
    physical address's data by adding or subtracting 1
    based on the value in the addressVector.
    """
    #Loop through the sdm's array of addresses:
    for i in range(self.p):

        #Compute the Hamming distance
        #between the address vector and the current physical address:
        hdist = hamming_distance(self.addresses[i], addressVector)

        #Check if the Hamming distance is within the radius:
        if hdist <= self.radius:
            #Update the data vector at this
            #physical address by looping through its bits:
            for j in range(self.n):
```

```
if addressVector[j] == 1:
    #If the bit in the address
    # vector is 1, add 1 to the data vector:
    self.data[i][j] += 1
else:
    #If the bit in the address
    # vector is 0, subtract 1 from the data vector:
    self.data[i][j] -= 1
```

Here is a graphical demonstration, that should hopefully make this process more clear:

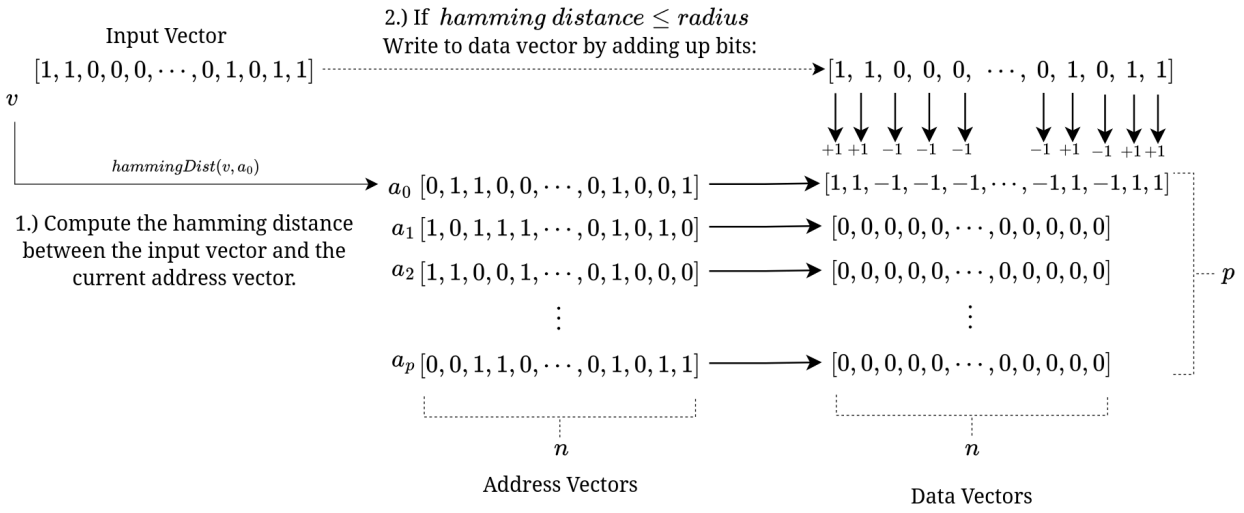

Figure 3: Writing to an SDM

2.3 Retrieving Data

To retrieve a pattern from the SDM, we provide a *query vector* and search for all hard locations whose stored *address labels* are within a fixed Hamming-distance radius of that query.This query vector acts as a sort of lookup key, and must be close enough to the input vector given a hamming distance comparison. For every address vector that is close enough to our query vector, we add its stored **data (vote) vector** into an accumulator vector of length n. This accumulator therefore contains, for each bit position, the *sum of votes* contributed by all nearby hard locations.

Finally, we convert the accumulated vote totals back into a binary output by applying a simple threshold: if the accumulated value at a bit position is nonnegative, we output 1; otherwise we output 0. Intuitively, this acts like a majority vote over the neighborhood, allowing the SDM to reconstruct a clean pattern from a noisy or incomplete query.

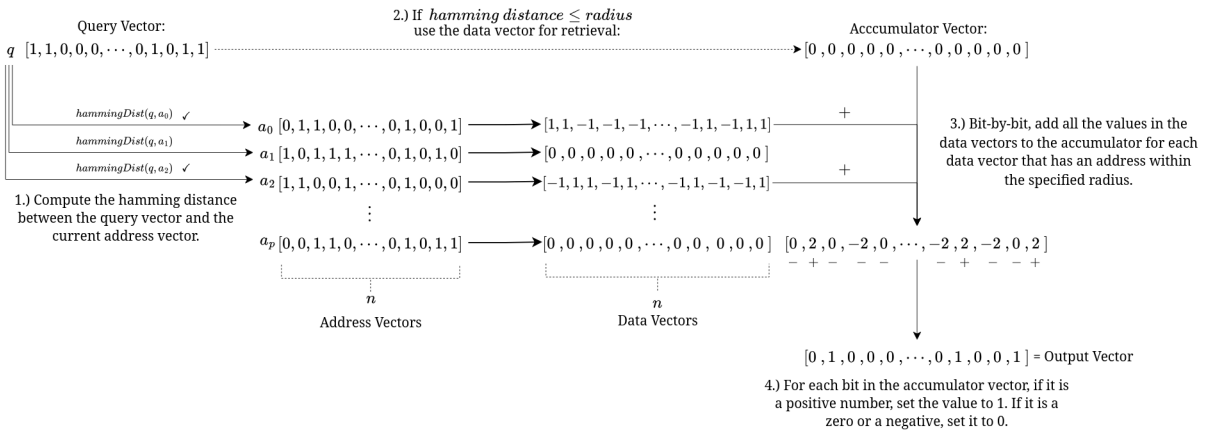

Figure 4: Retrieving from an SDM

Here it is in code:

```
def lookup(self, addressVector):
    """
    This method 'looks up' the information
    stored in the data vector
    of the sdm finding all the addresses
    within the neighborhood of the
    given address vector, and then
    adding each bit in the data to the retrieved data vector.
    The information stored in that retrieved vector is then converted to
    ones and zeros for output.
    """
    #Initialze array of 0s for retrieved data:
    retrieved_data = np.zeros(self.n)

    #Loop through the addresses:
    for i in range(self.p):

        #Compute the Hamming distance
        #between the address vector and the current physical address:
        hdist = hamming_distance(self.addresses[i], addressVector)

        #Check if the Hamming distance
        #is within the radius:
        if hdist <= self.radius:
            #If within the neighborhood,
            #add the data at the current address to our retrieved data vector:
            for j in range(self.n):
```

```
                retrieved_data[j] += self.data[i][j]

        #Convert the retrieved data to ones and zeros:
        for j in range(self.n):
            #If positive set to 1
            if retrieved_data[j] >= 0:
                retrieved_data[j] = 1
            #If negative set to zero:
            else:
                retrieved_data[j] = 0

        return retrieved_data
```

3 Proposed Obfuscation Architecture

To utilize the SDM as an obfuscator, we treat the malware payload not as a contiguous block of code, but as a series of memories to be recalled. The obfuscation process involves two distinct phases: the **Encoding Phase** (performed by the attacker) and the **Reconstruction Phase** (performed by the loader on the target system).

3.1 Encoding Phase

The target executable is sliced into k chunks, where the size of each chunk corresponds to the SDM vector dimension n. For each payload chunk P_i, a random address vector A_i is generated. This A_i serves as the "key" for that specific chunk. The system then writes P_i into the SDM using A_i as the address.

Crucially, the original payload P is discarded. The malware file distributed to the victim consists of two components:

1. The **serialized SDM structure** (the hard address matrix and the accumulated data counters).
2. The **sequence of retrieval keys** $(A_1, A_2, \ldots, A_k)$.

To a static analyst, the SDM structure appears as high-entropy statistical data, and the keys appear as random noise. There is no visible executable code.

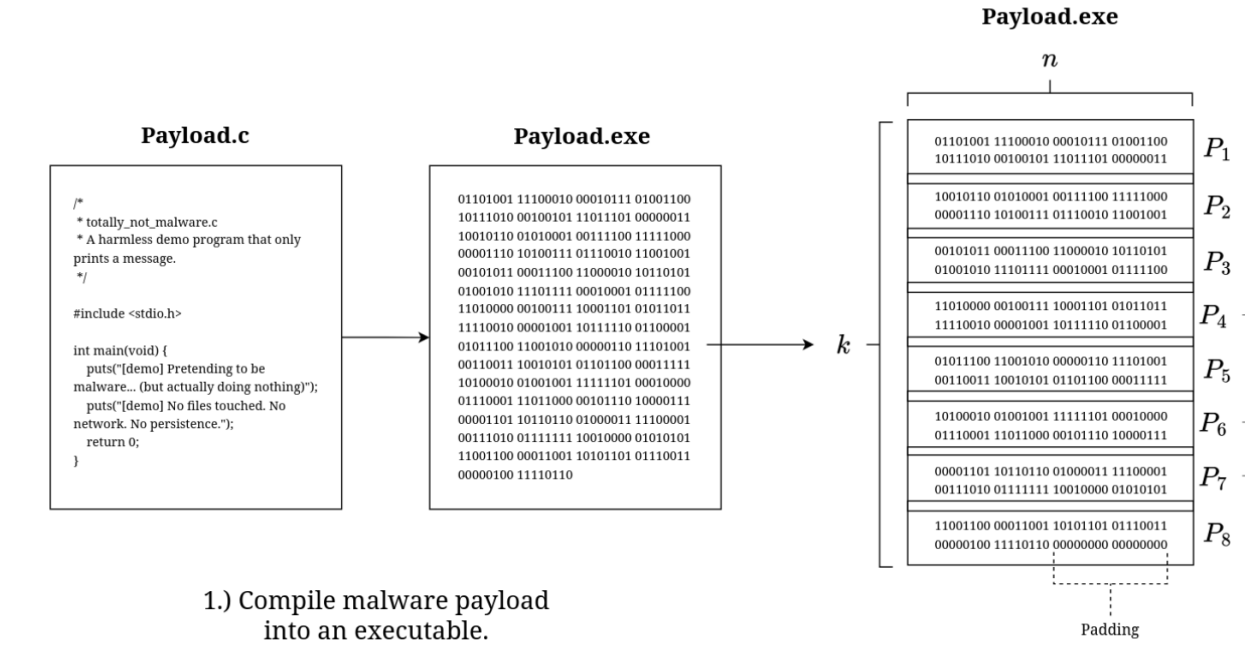

Figure 5: First, divide the payload into chunks and insert padding if needed.

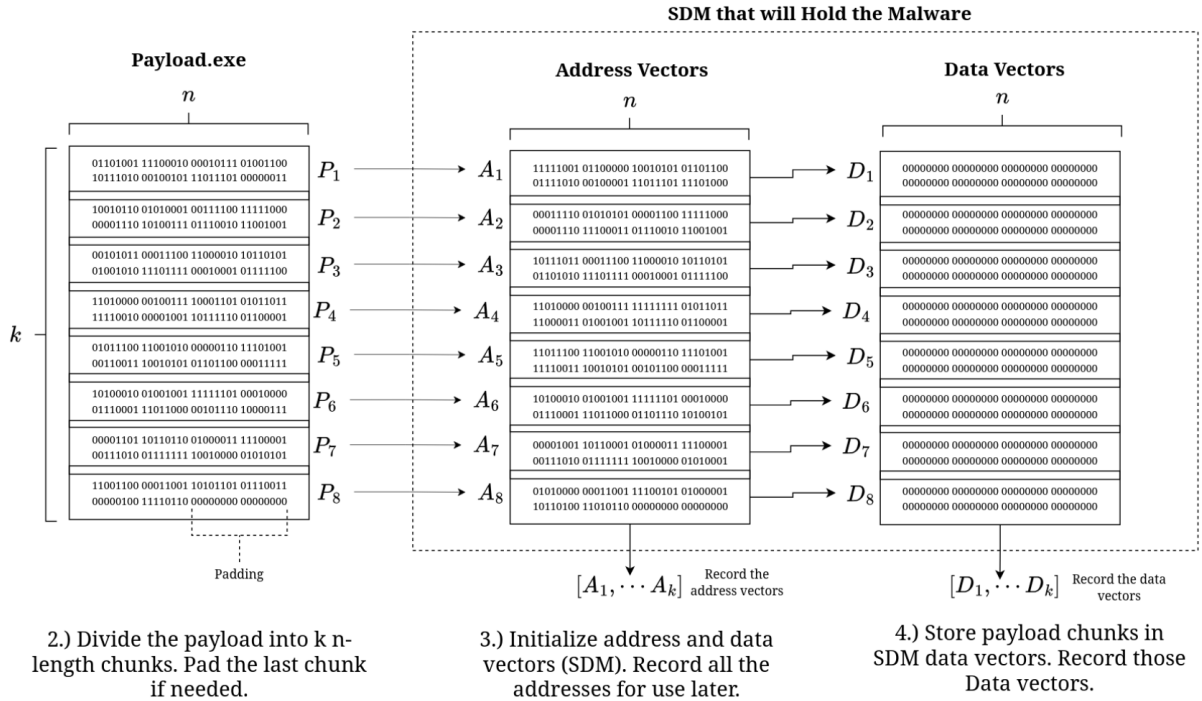

Figure 6: Then, store the payload into the sdm.

3.2 Reconstruction Phase

During reconstruction, the system does *not* need a local copy of the original data. Instead, it rebuilds the data incrementally using a sequence of retrieval keys.

First, the loader allocates an empty output buffer. It then obtains retrieval keys one at a time (for example, from a configuration record, a removable medium, or a remote controller in a benign update/recovery setting). For each key A_i, the loader performs a lookup operation, `lookup`(A_i), which returns the reconstructed bit-vector for the ith chunk. The loader immediately appends this reconstructed chunk to the output buffer and proceeds to the next key.

Because the keys are processed sequentially, reconstruction can be performed as a *streaming* procedure: the loader only needs the current key (and the SDM state) to recover the next chunk. After all keys have been processed, the output buffer contains the full reconstructed byte stream. In practical implementations, integrity checks (e.g., chunk hashes) and error-correcting codes can be applied per chunk to detect or correct any reconstruction errors before the final buffer is accepted as valid.

3.3 Addressing Capacity and Noise

Since SDM is a lossy compression format, "crosstalk" between stored memories can introduce bit errors. Unlike images or audio, executable code cannot tolerate bit flips. To mitigate this, we must ensure that the number of stored chunks k remains significantly below the SDM's critical capacity (approximately $0.15 \times p$). Additionally, standard Error Correcting Codes (ECC) can be applied to chunks prior to storage, allowing the loader to correct minor retrieval errors caused by memory interference.

4 Conclusion

Sparse Distributed Memory presents an interesting surface for malware obfuscation. By smearing executable code across a distributed associative structure, the technique eliminates any contiguous byte sequence that a static signature scanner could match against. To a naive analyst, the serialized SDM artifact is indistinguishable from random high-entropy data, and the retrieval keys are equally opaque without knowledge of the underlying architecture. In that narrow sense, the approach achieves its core obfuscation goal.

The most significant practical constraint, however, is storage overhead. An SDM with p address-data vector pairs of length n requires on the order of $2 \times p \times n$ bits of storage for the structure alone, before the retrieval keys are considered. For a modest executable payload of even a few hundred kilobytes, the p and n values required to keep the number of stored chunks k well below the critical capacity threshold of approximately $0.15 \times p$ will produce an SDM structure that dwarfs the original payload by one or two orders of magnitude. This is not merely an inconvenience — a several-megabyte blob of high-entropy data delivered to a target system is itself a behavioral signal that a sufficiently sophisticated detection pipeline could flag. The obfuscation gains on the static analysis front are therefore partially offset by the anomalous size and entropy profile of the artifact on the delivery front.

Mitigating this tradeoff is an open problem. One direction worth exploring is the use of a seeded pseudorandom number generator to reconstruct the address matrix on the target system at runtime, rather than transmitting it in full. If both the attacker and the loader share a seed, the address matrix never needs to be serialized at all, and the distributed file shrinks to just the data vectors and the retrieval keys. This would substantially reduce the storage footprint and remove the most obvious forensic artifact. Combined with per-chunk error correcting codes to handle retrieval noise, such an architecture would bring SDM-based obfuscation closer to practical viability, though thorough evaluation against both static and dynamic analysis pipelines remains necessary before any strong evasion claims can be made. Ultimately, this work is less a finished system than a proof of concept — an attempt to ask whether the associative, distributed properties of SDM that make it interesting as a cognitive model might also make it useful as an obfuscation primitive. The idea emerged from coursework discussions surrounding SDM and neuromorphic computing, and if nothing else, it suggests that there may be fertile ground here for future researchers to build on and improve.